

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a76e165fb6e63ff02.jsonl`  
- SHA-256: `9aa1c207778356ed80e53a525cd086981e3ebe1e200c35794a3874cca3ecac41`  
- Source modified: `2026-06-13T19:15:05+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.199Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

```
FINDING: {"title": "Fusion person-cue features are computed even when store_candidates=False (fusion-only, not a twin regression)", "severity": "low", "file": "backend/pipeline/segmenters/trajecory_hybrid.py:203-206", "detail": "window_stats is built unconditionally before the `if store_candidates` guard. For the base twins this is harmless (pure hook). For FusionSegmenter with the person cue ON, `_enrich_candidate_stats` -> `_person_features` runs the GPU PersonDetector over each window's frame range (fusion_hybrid.py:85-107) even when store_candidates=False and even for windows that will later be gated out of the handoff. The served-gating result is still correct (gating reads the already-computed stats), and the NO-REGRESSION claim is about the twins (unaffected), but this means the default-OFF promise 'never touches the GPU' only holds while cue_flags.person is false -- with the cue on, GPU work happens regardless of store_candidates and regardless of Gate A pruning. This is wasted work, not incorrect output.", "fix": "If person-cue GPU cost matters: compute net_crossings (cheap, GPU-free) for all windows always, but defer/skip the person-feature computation for windows that store_candidates=False would discard, or compute person features only for windows that survive Gate A. Out of scope for the current NO-REGRESSION verification; flag for the P3 learned-fusion work."}
```

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajecory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:55.925Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:57.520Z)

```
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)  
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)  
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
```

```

153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---
173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()
176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
180             return []
181
182         points = runner.parse_trajectory_csv(csv_path)
183         logger.info("Parsed %d trajectory points (%d visible).",
184                     len(points), sum(1 for p in points if p.visible))
185
186         all_candidate_windows = runner.trajectory_to_action_windows(
187             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189             min_window_duration=min_window_duration, chunk_index=0,
190         )
191         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                     label, time.time() - t_start, len(all_candidate_windows))
193
194         if log_candidates:
195             for cw in all_candidate_windows:
196                 logger.info(f"[{label} rally
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])}] "
198                             f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200                             f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204         # segmenter adds net_crossings + person-cue features. Computed once, reused for
205         # persistence and the handoff gate below.
206         window_stats = [
207             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
208             idx_cfg)
209             for cw in all_candidate_windows
210         ]
211
212         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
213         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
214         if store_candidates:
215             for i, cw in enumerate(all_candidate_windows):

```

```

212         candidate_ids[i] = self.db.add_candidate_segment(
213             video_id, detector=self.name,
214             start_time=cw["start"], end_time=cw["end"],
215             chunk_index=cw["chunk_index"], confidence=min(1.0,
216             cw["peak_velocity"]),
217             stats=window_stats[i], detection_params=detection_params,
218         )
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
224     # candidates above stay the full superset ? only the served play_segments are
225     # gated.
226     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
227     idx_cfg)
228     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
229     os.makedirs(handoff_dir, exist_ok=True)
230     logger.info("Phase 3: %s validation on %d candidate windows...",
231     provider_name, len(handoff_indices))
232
233     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
234         w_start = max(0, window["start"] - handoff_padding)
235         w_end = min(video_duration, window["end"] + handoff_padding)
236         w_dur = w_end - w_start
237         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
238         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
239         reencode=True):
240             return []
241         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
242         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
243         label=f"Handoff {wi+1}")
244         valid = []
245         for seg in segments:
246             try:
247                 seg["start_time"] = float(seg["start_time"]) + w_start
248                 seg["end_time"] = float(seg["end_time"]) + w_start
249                 seg["_candidate_id"] = candidate_ids[wi]
250                 valid.append(seg)
251             except (ValueError, TypeError, KeyError) as e:
252                 logger.warning("Dropping segment with unparseable timestamps: %s ?
253         %s", seg, e)
254         try:
255             os.remove(w_path)
256         except OSError:
257             pass
258         return valid
259
260     ai_segments: List[dict] = []
261     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
262     executor:
263         futures = [executor.submit(process_candidate_window, wi,
264         all_candidate_windows[wi])
265         for wi in handoff_indices]
266         for future in futures:
267             ai_segments.extend(future.result())
268     try:
269         os.rmdir(handoff_dir)
270     except OSError:
271         pass
272     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
273     len(ai_segments))
274

```

```
268         # --- Phase 4: DB population + candidate linking ---
269         segments_data = []
```

4. user (2026-06-13T19:13:57.743Z)

```
1         """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3         Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4         TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5         do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6         shuttle-seeded window via two overridable seams the engine exposes:
7
8         - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14        - ``_select_for_handoff`` ? optional served Gate A/B: when
15        ``indexing.fusion.min_crossings``
16        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17        are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
20        experiment
21        harness can re-score any variant.
22
23        Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24        but
25        nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26        seeds
27        and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28        default
29        segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30        through the
31        experiment harness (EXPERIMENT_HARNESS.md), never silently.
32        """
33        from typing import Any, Dict, List, Optional, Tuple
34
35        from backend.pipeline.segmenters.base import segmenter_registry
36        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37        from backend.pipeline.detectors.base import DetectorRunner
38        from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40        from backend.pipeline.detectors import rally_gate
41        from backend.pipeline.detectors import fusion_features as ff
42
43        class FusionSegmenter(TrajectoryHybridSegmenter):
44            """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45            B)."""
46
47            family = "fusion"
48            display_label = "Fusion"
49
50            def __init__(self, db: Any, config: Any):
51                super().__init__(db, config)
52                # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53                default path).
54                self._person: Optional[Any] = None
55
56            @property
57            def name(self) -> str:
58                return "fusion_hybrid"
```

```

52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                 "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None

```

```

104         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

5. assistant (2026-06-13T19:14:00.784Z)

Now let me look at the base `\_enrich\_candidate\_stats` hook in the parent class and the `\_select\_for\_handoff` default, plus the PersonDetector to confirm the GPU claim.

6. user (2026-06-13T19:14:03.281Z)

```

82:     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83:                                frame_width: float, video_path: str,
84:                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85:         """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
86:         returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
87:         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88:         features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
89:         return {
90:             "peak_velocity": cw["peak_velocity"],
91:             "mean_velocity": cw["mean_velocity"],
92:             "active_frame_count": cw["active_frame_count"],
93:             "track_id_count": cw["track_id_count"],
94:         }
95-
96:     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97:                           window_stats: List[Dict[str, Any]],
98:                           idx_cfg: Dict[str, Any]) -> List[int]:
99:         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100:         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101:         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102:         Persisted candidates are NOT affected ? they remain the full superset for offline
103:         re-scoring."""
104:         return list(range(len(windows)))
105-
106:     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107:                      player_pool: Optional[List[str]] = None,
108:                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:

```

```

109- # override-ignore: the ABC declares the Result contract (Success|Failure)
110- # but every live segmenter still returns a bare list ? the documented
111- # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112- label = self.display_label
113- # --- Sport profile + AI provider wiring (identical across segmenters) ---
114- sport_name = self.config.get("sport", "badminton")
115- try:
116-     sport_profile = sport_registry.get(sport_name)
117- except KeyError as e:
118-     logger.error(str(e))
119-     return []
120-
121- idx_cfg = self.config.get("indexing", {})
122- --
204-     self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
205-     for cw in all_candidate_windows
206- ]
207-
208- # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
209- candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
210- if store_candidates:
211-     for i, cw in enumerate(all_candidate_windows):
212-         candidate_ids[i] = self.db.add_candidate_segment(
213-             video_id, detector=self.name,
214-             start_time=cw["start"], end_time=cw["end"],
215-             chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
216-             stats=window_stats[i], detection_params=detection_params,
217-         )
218-
219- if not all_candidate_windows:
220-     return []
221-
222- # --- Phase 3: AI provider handoff over padded candidate clips ---
223- # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
224- # candidates above stay the full superset ? only the served play_segments are gated.
225- handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226- handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227- os.makedirs(handoff_dir, exist_ok=True)
228- logger.info("Phase 3: %s validation on %d candidate windows...",
229-             provider_name, len(handoff_indices))
230-
231- def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232-     w_start = max(0, window["start"] - handoff_padding)
233-     w_end = min(video_duration, window["end"] + handoff_padding)
234-     w_dur = w_end - w_start
235-     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236-     if not extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
237-         return []
238-     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239-     segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240-                                         label=f"Handoff {wi+1}")
241-     valid = []
242-     for seg in segments:
243-         try:
244-             seg["start_time"] = float(seg["start_time"]) + w_start
245-             seg["end_time"] = float(seg["end_time"]) + w_start
246-             seg["_candidate_id"] = candidate_ids[wi]
247-             valid.append(seg)
248-         except (ValueError, TypeError, KeyError) as e:
249-             logger.warning("Dropping segment with unparseable timestamps: %s ? %s",
seg, e)
250-     try:

```

```

1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3      Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4      the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5      `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
6      identically);
7      next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
8      later the learned classifier reads its persisted features. Keeping detection here (not
9      in
10     the segmenter) is what makes "add a cue = register a producer" true.
11
12     Behaviour is unchanged from the in-lined version ? only the home moved:
13     - `lazy_load_model`           ? load a permissively-licensed torchvision detector
14     - `make_tracker`             ? a fresh supervision ByteTrack per chunk
15     - `detect_and_track`         ? one frame ? confident persons with persistent IDs
16     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
17         with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
18         that feed the candidate `stats` JSON and the learned fusion features.
19
20     Heavy deps (torchvision, supervision) are imported lazily inside the methods so
21     importing
22     the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
23     the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
24     (ADR-005; ultralytics' AGPL YOLO was dropped).
25     """
26     import logging
27     from typing import Any, Dict, List, Optional, Tuple
28
29     import cv2
30     import torch
31
32     from backend.utils.geometry import get_velocity_normalised
33
34     logger = logging.getLogger(__name__)
35
36     # torchvision detection models are trained on COCO where the "person" category is
37     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
38     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
39     _PERSON_CLASS_ID = 1
40
41     # Permissively-licensed torchvision detectors selectable via config
42     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
43     _DETECTOR_FACTORIES = {
44         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
45         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
46         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
47     }
48     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
49
50     class PersonDetector:
51         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55         min_action_window_duration}``); the model loads lazily on first use.
56         """
57
58         def __init__(self, config: Dict[str, Any]):
59             self.config = config
60             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61             # `self.model([tensor])` inference call type-checks without a quarantine.
62             self.model: Any = None
63             self.device: Optional[str] = None

```



```

63     def lazy_load_model(self) -> None:
64         if self.model is not None:
65             return
66
67         # Imported lazily so the module imports without torchvision present and so
68         # the test suite (which pre-sets self.model) never pulls in model weights.
69         from torchvision.models import detection as tv_detection
70
71         idx_cfg = self.config.get("indexing", {})
72         use_gpu = idx_cfg.get("use_gpu", True)
73         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74         if use_gpu and not torch.cuda.is_available():
75             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78         if model_name not in _DETECTOR_FACTORIES:
79             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80             model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97
98     def detect_and_track(self, frame: Any, tracker: Any,
99                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
112         with torch.no_grad():
113             outputs = self.model([tensor])
114
115         out = outputs[0]
116         boxes = out["boxes"].detach().cpu().numpy()
117         labels = out["labels"].detach().cpu().numpy()
118         scores = out["scores"].detach().cpu().numpy()
119
120         # Keep only confident person detections.
121         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122         if not mask.any():
123             return []

```

```

124
125         # ByteTrack requires confidence to be populated (it filters on it internally).
126         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127         tracked = tracker.update_with_detections(dets)
128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                         velocity_thresh: float, merge_gap: float,
140                                         frame_skip: int = 3,
141                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142         """
143         Runs person detection + tracking on a video chunk and extracts high-motion
144         action windows. Returns a list of dicts in the full video's timeframe, each
with:
145             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146             track_id_count, chunk_index.
147
148         Args:
149             frame_skip: Process every Nth frame (1 = every frame). Higher values
150                         dramatically reduce inference cost with minimal quality loss.
151             chunk_index: Index of the source chunk (recorded on each window for
traceability).
152         """
153         cap = cv2.VideoCapture(chunk_path)
154         if not cap.isOpened():
155             logger.error(f"Could not open {chunk_path}")
156             return []
157
158         fps = cap.get(cv2.CAP_PROP_FPS)
159         if fps <= 0:
160             fps = 30.0
161
162         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163         if frame_width <= 0:
164             frame_width = 1280.0 # Sensible fallback
165
166         # Effective sampling rate after frame-skipping
167         effective_fps = fps / max(frame_skip, 1)
168
169         # A fresh tracker per chunk ? track IDs only need to be consistent within a
170         # chunk (windows are computed per chunk, then merged across chunks by time).
171         tracker = self.make_tracker(effective_fps)
172         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175         # so windows can be enriched with motion stats for logging + fine-tuning.
176         active_frames = []
177         frame_idx = 0
178         track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180         while True:
181             ret, frame = cap.read()
182             if not ret:
183                 break

```

```

184
185     # Skip frames for performance ? at 30fps, every 3rd frame still
186     # gives ~10 samples/second, plenty for human-scale velocity.
187     if frame_idx % max(frame_skip, 1) != 0:
188         frame_idx += 1
189         continue
190
191     # Detect persons + assign persistent track IDs.
192     detections = self.detect_and_track(frame, tracker, score_thresh)
193
194     frame_peak_velocity = 0.0
195     frame_active_ids = set()
196     current_ids = set()
197
198     for track_id, bbox in detections:
199         current_ids.add(track_id)
200
201         if track_id in track_history:
202             prev_bbox = track_history[track_id]
203             velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204             if velocity > velocity_thresh:
205                 frame_peak_velocity = max(frame_peak_velocity, velocity)
206                 frame_active_ids.add(track_id)
207
208             track_history[track_id] = bbox
209
210     # Prune stale IDs that are no longer visible ? prevents false
211     # velocity spikes when the tracker reassigns an old ID.
212     track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214     if frame_active_ids:
215         active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217     frame_idx += 1
218
219 cap.release()
220
221 # Group active frames into continuous windows
222 if not active_frames:
223     return []
224
225 max_gap_frames = int(merge_gap * fps)
226
227 def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228     velocities = [v for _, v, _ in group]
229     track_ids = set().union(*[ids for _, _, ids in group])
230     return {
231         "start": group[0][0] / fps + chunk_start,
232         "end": group[-1][0] / fps + chunk_start,
233         "active_frame_count": len(group),
234         "peak_velocity": max(velocities),
235         "mean_velocity": sum(velocities) / len(velocities),
236         "track_id_count": len(track_ids),
237         "chunk_index": chunk_index,
238     }
239
240 windows = []
241 group = [active_frames[0]]
242 for af in active_frames[1:]:
243     if af[0] - group[-1][0] <= max_gap_frames:
244         group.append(af)
245     else:
246         windows.append(_finalize(group))
247         group = [af]

```

```

248         windows.append(_finalize(group))
249
250         # Filter out extremely short action windows
251         min_window_duration = self.config.get("indexing",
252         {}).get("min_action_window_duration", 2.0)
253         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254         return windows
255
256     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257                             frame_skip: int = 3) -> Dict[str, Any]:
258         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
259         video.
260
261         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
262         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
263         discards the
264         per-frame boxes), this returns the raw per-frame detections so the fusion
265         features can
266         align them with the shuttle trajectory. It reads the source by **absolute frame
267         index**
268         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
269         match
270         the WASB trajectory's ``.frame`` exactly ? that alignment is what
271         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
272
273         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
274         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
275         seek
276         accuracy varies by codec (it lands near the requested keyframe then decodes
277         forward);
278         treat the indices as ±a few frames on real footage ? fine for window-level
279         features.
280
281         """
282         self.lazy_load_model()
283         cap = cv2.VideoCapture(video_path)
284         if not cap.isOpened():
285             logger.error(f"Could not open {video_path}")
286             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
287
288         fps = cap.get(cv2.CAP_PROP_FPS)
289         if fps <= 0:
290             fps = 30.0
291         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
292         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
293         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
294         0.5)
295
296         step = max(frame_skip, 1)
297         tracker = self.make_tracker(fps / step)
298
299         start_frame = max(0, int(start_frame))
300         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
301         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
302         frame_idx = start_frame
303         while frame_idx <= end_frame:
304             ret, frame = cap.read()
305             if not ret:
306                 break
307             if (frame_idx - start_frame) % step == 0:
308                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
309                 frame_idx += 1
310         cap.release()
311         return {"frames": frames, "fps": fps,
312                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
313

```

8. assistant (2026-06-13T19:14:12.763Z)

The finding's core claims are confirmed by the code. Let me verify the precise claims:

1. ``window_stats`` is built unconditionally at lines 203-206, before the ``if store_candidates`` guard at line 210. ****Confirmed.****
2. The list comprehension calls ``_enrich_candidate_stats`` for every window. ****Confirmed.****
3. In ``FusionSegmenter._enrich_candidate_stats`` (`fusion_hybrid.py:81-82`), when ``cue_flags.person`` is True, it calls ``_person_features``, which (lines 92-98) lazily builds ``PersonDetector`` and calls ``detections_per_frame``, which (`person_detector.py:273`) calls ``lazy_load_model()`` and runs torchvision inference. ****Confirmed GPU work happens.****
4. This happens regardless of ``store_candidates`` (computed before the guard). ****Confirmed.****
5. This happens regardless of Gate A pruning, because ``_select_for_handoff`` (line 225) runs AFTER ``window_stats`` is already fully computed, and it reads the already-computed stats. ****Confirmed.****

Let me check the config model to verify the default-OFF claim for ``cue_flags.person``, and confirm `net_crossings` is GPU-free as the fix suggests.

9. user (2026-06-13T19:14:14.467Z)

```
75-     min_free_gb: float = 0.0
76-
77-
78: class FusionConfig(BaseModel):
79:     """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
80-
81:     Every default reproduces control behaviour: the fusion segmenter persists the
82:     shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
83:     true ? the person-cue features, but it does NOT gate the served handoff unless a
84:     threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
85:     is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
86:     supplied ? off-court persons (spectators / adjacent courts) are excluded.
87:     """
88:     # Which trajectory substrate seeds the windows (shuttle stays primary).
89:     substrate: Literal["wasb", "tracknet"] = "wasb"
90:     # Windowing velocity threshold for the fusion family (the engine reads
91:     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92:     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93:     velocity_threshold: float = 0.02
94:     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95:     # enabled ? so the default path never imports torch / touches the GPU.
96:     cue_flags: dict[str, bool] = Field(default_factory=dict)
97:     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
98:     # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
99:     # scoring).
100:     min_crossings: int = Field(default=0, ge=0)
101:     # Served Gate B: when the person cue is on, drop windows with median in-court players
102:     # < this. 0 = OFF.
103:     min_players: int = Field(default=0, ge=0)
104:     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
105:     court_polygon: Optional[list[list[float]]] = None
106:     # Net line for the person two-sided-motion split (person features only ? the shuttle
107:     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
108:     net_axis: Literal["x", "y"] = "y"
109:     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
110:     --
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141-     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142-     fusion: FusionConfig = Field(default_factory=FusionConfig)
143-
144-     @field_validator("default_segmenter")
145-     @classmethod
```

10. assistant (2026-06-13T19:14:31.618Z)

All claims confirmed. ``cue_flags`` defaults to empty dict (line 96), so ``person`` is OFF by default ? the default-OFF promise holds. The finding correctly notes that the "never touches the GPU" guarantee only holds while ``cue_flags.person`` is false.

Now let me verify the finding's claim about ordering precisely ? that the wasted work is genuine and not somehow short-circuited. The key sequence in ``process_video``:

- Line 203-206: ``window_stats`` computed for **all** windows unconditionally (this triggers ``_person_features`` ? GPU when person cue on).
- Line 210: ``if store_candidates`` guard ? only controls DB persistence, not the stats computation.
- Line 225: ``_select_for_handoff`` (Gate A/B) runs after, reading already-computed stats.

So the GPU work in ``_person_features`` does happen for every window regardless of ``store_candidates`` and regardless of Gate A pruning. The served output is still correct (gating reads the stats). This is wasted compute, not incorrect output. Let me confirm ``rally_gate.gate_windows`` (``net_crossings``) is indeed GPU-free as the fix suggests.

11. user (2026-06-13T19:14:33.330Z)

```
29:class GatedWindow:
91:def gate_windows(points, windows: List[Interval], fps: float,
```

12. assistant (2026-06-13T19:14:45.670Z)

``rally_gate.py`` imports no torch/cv2/torchvision ? confirming ``net_crossings`` is GPU-free (operates on the already-parsed trajectory ``points``). The fix's premise (``net_crossings`` is cheap/GPU-free, person features are the expensive GPU part) is accurate.

I have fully verified the finding against the code. Let me assess severity and validity.

****Verification summary:****

The finding is factually accurate on every load-bearing claim:

- ``window_stats`` (``trajectory_hybrid.py``:203-206) is computed unconditionally before the ``if store_candidates`` guard (line 210).
- For ``FusionSegmenter`` with ``cue_flags.person=True``, ``_enrich_candidate_stats`` ? ``_person_features`` (``fusion_hybrid.py``:81-82, 85-107) lazily builds ``PersonDetector`` and calls ``detections_per_frame``, which calls ``lazy_load_model()`` + runs torchvision inference (``person_detector.py``:273, 297) ? genuine GPU work.
- This runs for **every** candidate window, before ``store_candidates`` is checked and before ``_select_for_handoff``/Gate-A pruning (line 225) ? so the GPU cost is incurred regardless of both.
- The served output is still correct (gating reads already-computed stats); this is wasted compute, not a correctness bug.
- The default-OFF promise genuinely holds: ``cue_flags`` defaults to ``{}`` (``config/models.py``:96), so the GPU is never touched on the default path. The finding explicitly acknowledges this.

****Severity assessment:**** This is correctly classified as ``low``. It is not a regression in the twins (their ``_enrich_candidate_stats`` returns 4 pure-Python motion keys, no GPU). It is not incorrect output. It is a wasted-work / efficiency observation that only manifests on an explicitly opt-in, non-default code path (person cue ON), and the finding itself scopes it out of the current NO-REGRESSION verification and flags it for P3. The fix is sensible and correctly identifies ``net_crossings`` as the cheap GPU-free signal to keep eager.

It is a real, code-grounded efficiency observation rather than a false positive or a fabricated nitpick ? but it is genuinely minor and self-scoped-as-deferred.

13. user (2026-06-13T19:14:59.564Z)

Structured output provided successfully

14. assistant (2026-06-13T19:15:05.293Z)

Verified the finding against the actual code. It holds: real=true, severity=low.

Key evidence: ``window_stats`` is built unconditionally at ``trajectory_hybrid.py:203-206`` (before the ``store_candidates`` guard at line 210 and before ``_select_for_handoff`` Gate-A pruning at line 225), and ``FusionSegmenter._enrich_candidate_stats`` ? ``_person_features`` (fusion_hybrid.py:81-82, 85-107) runs real GPU torchvision inference via ``PersonDetector.detections_per_frame`` ? ``lazy_load_model()`` (person_detector.py:273, 297) when ``cue_flags.person`` is on. So the GPU work happens for every window regardless of ``store_candidates`` and regardless of which windows Gate A would discard ? wasted compute, not incorrect output. The default path is unaffected (``cue_flags`` defaults to ``{}``, config/models.py:96) and the twins are unaffected (base hook returns 4 pure-Python keys). Correctly scoped as a low-severity, deferred-to-P3 efficiency note rather than a regression.